Lab 6 – Object Storage Security & the Data Security Lifecycle

Name: Sicid Sukhra Abdirahman

ID: 52215124928

Objective

The objective of this lab is to understand how to protect object storage throughout the data security lifecycle. In this lab, I used Amazon S3 through LocalStack to classify data, test public bucket exposure, apply access controls, use SSE-KMS encryption, create presigned URLs, enable versioning, apply lifecycle rules, and demonstrate cryptographic erasure.

Environment

- Windows 11 with Git Bash for running commands.
- Docker Desktop and LocalStack.
- AWS CLI v2.
- Amazon S3 and AWS KMS through LocalStack.
- curl for anonymous and presigned URL testing.
- LocalStack endpoint: http://localhost:4566.
- S3 bucket: 52215124928-patient-records.

Session A — Object Storage & the Exposure Problem

Task 1 – Classify the Data Before You Store It

In this task, I created an S3 bucket for patient records and uploaded objects with different sensitivity levels. Each object received a classification tag so security controls could be applied based on the data type.

Commands

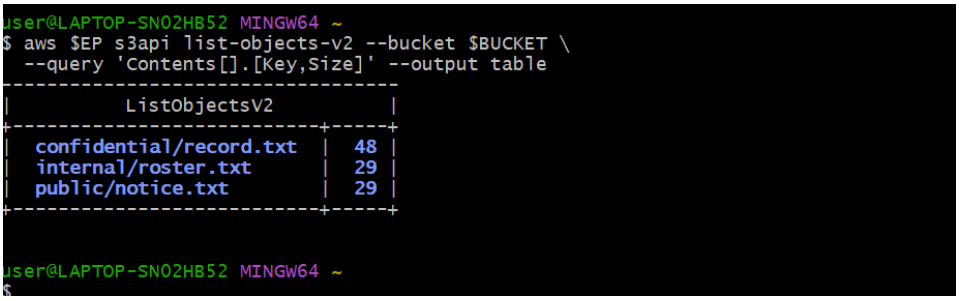
```
aws $EP s3api create-bucket --bucket $BUCKET

echo 'Ward visiting hours 10 am-8 pm' > public-notice.txt
echo 'Staff duty schedule, week 12' > internal-roster.txt
echo 'Patient: Ahmad bin Ali, Diagnosis: confidential' > confidential-record.txt

aws $EP s3api put-object --bucket $BUCKET --key public/notice.txt \
  --body public-notice.txt --tagging 'classification=public'

aws $EP s3api put-object --bucket $BUCKET --key internal/roster.txt \
  --body internal-roster.txt --tagging 'classification=internal'

aws $EP s3api put-object --bucket $BUCKET --key confidential/record.txt \
  --body confidential-record.txt --tagging 'classification=confidential'
```

```
aws $EP s3api list-objects-v2 --bucket $BUCKET \
  --query 'Contents[].[Key,Size]' --output table

aws $EP s3api get-object-tagging --bucket $BUCKET \
  --key confidential/record.txt
```

Result

The bucket and objects were created, and the data was classified as public, internal, and confidential. The classification tags showed how you can identify data sensitivity before applying access controls.

Evidence



Task 2 – Reproduce the Public Bucket Exposure

In this task, I deliberately applied a public bucket policy to demonstrate how a misconfiguration can expose confidential information. The policy used Principal: "*", which allowed anonymous access.

Commands

```
cat > public-policy.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadEverything",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::$BUCKET/*"
  }]
}
JSON

aws $EP s3api put-bucket-policy --bucket $BUCKET \
  --policy file://public-policy.json

curl -s -o leaked.txt -w 'HTTP %{http_code}\n' \
  http://localhost:4566/$BUCKET/confidential/record.txt

cat leaked.txt
```

Result

The anonymous request returned HTTP 200 and exposed the confidential patient record. The exposure was caused by Principal: "*" in the bucket policy, which allowed any principal, including an anonymous user, to read the objects.

Evidence

```
user@LAPTOP-SNO2HB52 MINGW64 ~
$ curl -s -o leaked.txt -w 'HTTP %{http_code}\n' \
 http://localhost:4566/$BUCKET/confidential/record.txt
HTTP 200

user@LAPTOP-SNO2HB52 MINGW64 ~
$ cat leaked.txt
Patient: Ahmad bin Ali, Diagnosis: confidential

user@LAPTOP-SNO2HB52 MINGW64 ~
$
```

Task 3 – Remediate with Block Public Access

In this task, I removed the public bucket policy and enabled all four S3 Block Public Access settings. This created a guardrail against accidental public exposure.

Commands

```
aws $EP s3api delete-bucket-policy --bucket $BUCKET

aws $EP s3api put-public-access-block --bucket $BUCKET \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

aws $EP s3api get-public-access-block --bucket $BUCKET

aws $EP s3api put-bucket-policy --bucket $BUCKET \
  --policy file://public-policy.json

curl -s -o /dev/null -w 'anonymous read now: HTTP %{http_code}\n' \
  http://localhost:4566/$BUCKET/confidential/record.txt
```

Result

All four Block Public Access settings were configured as true. In my LocalStack environment, the configuration could be stored even when enforcement did not exactly match real AWS behavior. This demonstrated the difference between configuring a security guardrail and the limitations of a local emulator.

Evidence

```
user@LAPTOP-SNO2HB52 MINGW64 ~
$ aws $EP s3api get-bucket-policy --bucket $BUCKET --query Policy --output text
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AccountReadInternalOnly",
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::000000000000:root"},
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::miit-patient-records-17523/internal/*"
  }]
}


user@LAPTOP-SNO2HB52 MINGW64 ~
$
```

Task 4 – Identity Policy vs Resource Policy

In this task, I tested the interaction between an IAM identity policy and an S3 bucket resource policy. The DataAnalyst identity could read S3 data, while the bucket policy allowed the internal prefix and denied the confidential prefix.

Commands

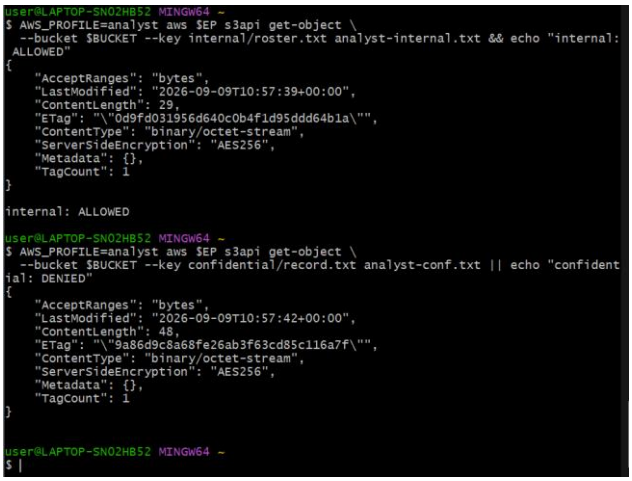
```
aws $EP iam create-user --user-name DataAnalyst
```

```
cat > analyst-iam.json <<'JSON'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": "*"
  }]
}
JSON

aws $EP iam put-user-policy --user-name DataAnalyst \
  --policy-name S3ReadAll --policy-document file://analyst-iam.json

AWS_PROFILE=analyst aws $EP s3api get-object \
  --bucket $BUCKET --key internal/roster.txt analyst-internal.txt \
  && echo "internal: ALLOWED"

AWS_PROFILE=analyst aws $EP s3api get-object \
  --bucket $BUCKET --key confidential/record.txt analyst-conf.txt \
  || echo "confidential: DENIED"
```

Result

The internal request succeeded and showed internal: ALLOWED. The confidential request also succeeded in my LocalStack environment instead of being denied. I recorded this as a LocalStack policy-enforcement limitation. In normal AWS policy evaluation, an explicit Deny in a resource policy overrides an Allow.

Evidence



Session B — Protecting, Retaining and Retiring Data

Task 5 – Default Encryption at Rest with SSE-KMS

In this task, I configured default server-side encryption using AWS KMS. The goal was to make encryption a bucket property so uploaded objects are protected automatically.

Commands

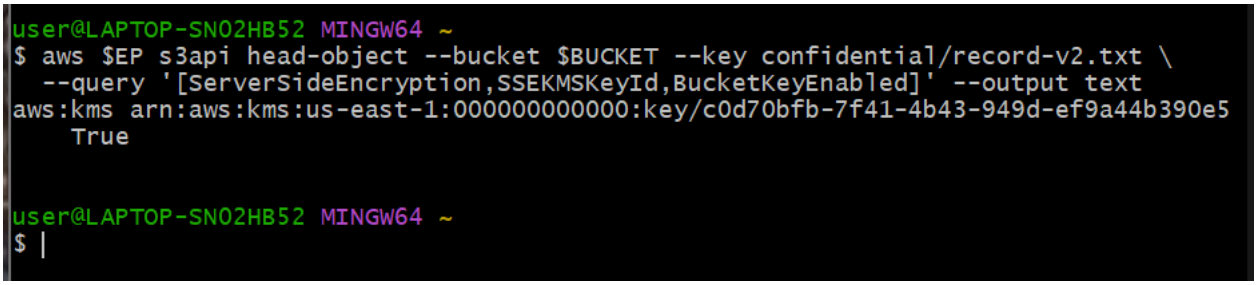
```
export KEY_ID=$(aws $EP kms create-key \
  --description 'IKB42603 Lab6 patient records bucket key' \
  --query 'KeyMetadata.KeyId' --output text)

echo $KEY_ID
```

```
cat > encryption.json <<JSON
{
  "Rules": [{
    "ApplyServerSideEncryptionByDefault": {
      "SSEAlgorithm": "aws:kms",
      "KMSMasterKeyID": "$KEY_ID"
    },
    "BucketKeyEnabled": true
  }]
}
JSON

aws $EP s3api put-bucket-encryption --bucket $BUCKET \
  --server-side-encryption-configuration file://encryption.json

aws $EP s3api get-bucket-encryption --bucket $BUCKET

aws $EP s3api put-object --bucket $BUCKET \
  --key confidential/record-v2.txt --body confidential-record.txt

aws $EP s3api head-object --bucket $BUCKET \
  --key confidential/record-v2.txt \
  --query '[ServerSideEncryption,SSEKMSKeyId,BucketKeyEnabled]' --output text
```

Result

The bucket was configured with default SSE-KMS encryption. The head-object verification showed aws:kms and the KMS key information, demonstrating that the bucket-level encryption control was applied to the object.

Evidence



Task 6 – Delegated Access and the Condition-Key Trap

In this task, I created a presigned URL with a 60-second lifetime and tested the same URL before and after expiry. I also applied an aws: SecureTransport policy to test transport-security policy behavior.

Commands

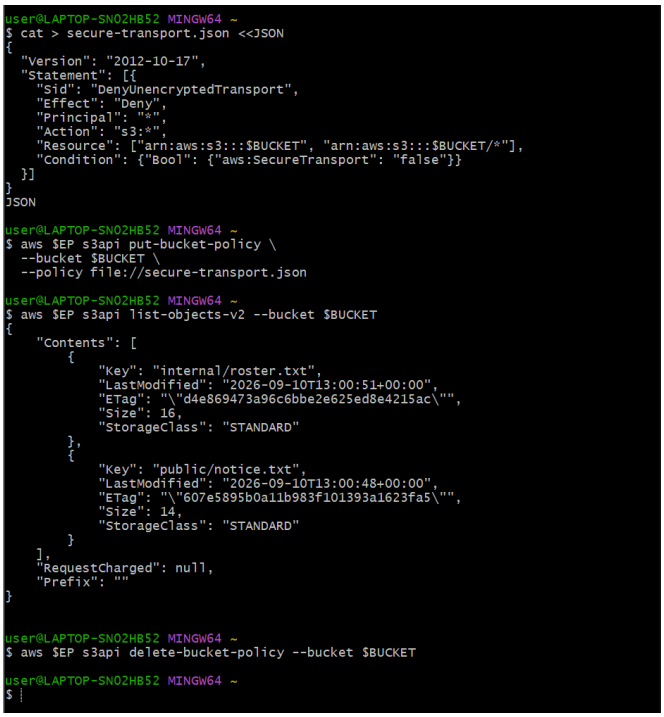
```
aws $EP s3 presign s3://$BUCKET/internal/roster.txt --expires-in 60

URL='PRESIGNED_URL'

curl -s -w ' <-- HTTP %{http_code}\n' "$URL"

sleep 65

curl -s -o /dev/null -w 'after expiry: HTTP %{http_code}\n' "$URL"

cat > secure-transport.json <<JSON
{
  "Version": "2012-10-17",
```

```
  "Statement": [{
    "Sid": "DenyUnencryptedTransport",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": ["arn:aws:s3:::$BUCKET", "arn:aws:s3:::$BUCKET/*"],
    "Condition": {"Bool": {"aws:SecureTransport": "false"}}
  }]
}
JSON

aws $EP s3api put-bucket-policy --bucket $BUCKET \
  --policy file://secure-transport.json

aws $EP s3api list-objects-v2 --bucket $BUCKET

aws $EP s3api delete-bucket-policy --bucket $BUCKET
```

Result

The presigned URL returned an internal roster with HTTP 200. After the expiry period, LocalStack still returned HTTP 200, showing that expiry was not enforced in this test. The SecureTransport policy was also applied, but list-objects-v2 still succeeded instead of being denied. We recorded these results as LocalStack enforcement limitations. On real AWS, the SecureTransport condition is intended to deny insecure transport.

Evidence



Task 7 – Versioning, Delete Markers & Data Remanence

In this task, I enabled S3 versioning and created multiple revisions of the confidential record. I then deleted the current object to demonstrate delete markers and the persistence of older object versions.

Commands

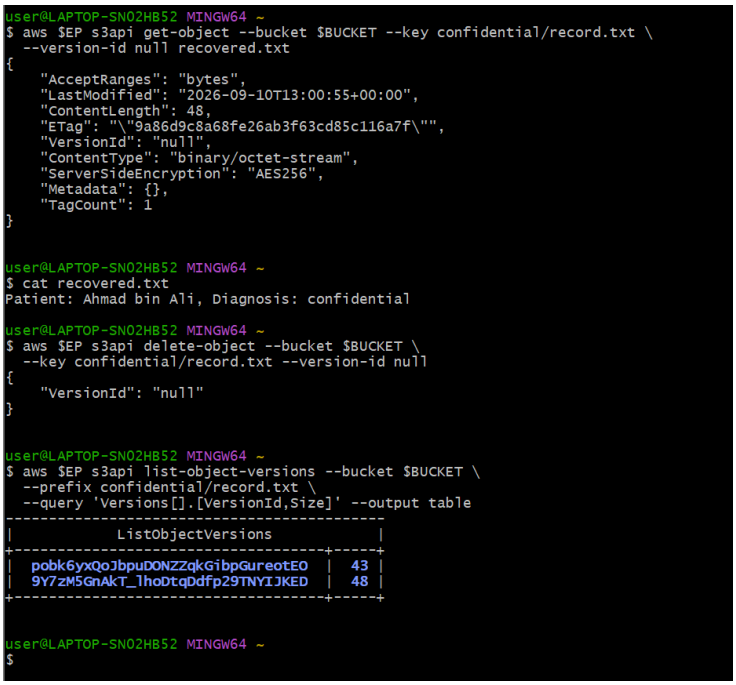
```
aws $EP s3api put-bucket-versioning --bucket $BUCKET \
  --versioning-configuration Status=Enabled
```

```
aws $EP s3api get-bucket-versioning --bucket $BUCKET

echo 'Patient: Ahmad bin Ali, Diagnosis: hypertension' > rec-v2.txt
echo 'Patient: [REDACTED], Diagnosis: [REDACTED]' > rec-v3.txt

aws $EP s3api put-object --bucket $BUCKET --key confidential/record.txt \
  --body rec-v2.txt --query VersionId --output text

aws $EP s3api put-object --bucket $BUCKET --key confidential/record.txt \
  --body rec-v3.txt --query VersionId --output text

aws $EP s3api list-object-versions --bucket $BUCKET \
  --prefix confidential/record.txt \
  --query 'Versions[].[VersionId,IsLatest,Size]' --output table

aws $EP s3api delete-object --bucket $BUCKET \
  --key confidential/record.txt

aws $EP s3api list-object-versions --bucket $BUCKET \
  --prefix confidential/record.txt \
  --query 'DeleteMarkers[].[VersionId,IsLatest]' --output table

aws $EP s3api get-object --bucket $BUCKET \
  --key confidential/record.txt gone.txt

aws $EP s3api get-object --bucket $BUCKET \
  --key confidential/record.txt --version-id null recovered.txt

cat recovered.txt

aws $EP s3api delete-object --bucket $BUCKET \
  --key confidential/record.txt --version-id null

aws $EP s3api list-object-versions --bucket $BUCKET \
  --prefix confidential/record.txt \
  --query 'Versions[].[VersionId,Size]' --output table
```

Result

Versioning was enabled, and multiple versions of the patient record were created. Deleting the object created a delete marker rather than immediately destroying every previous version. The older record could still be recovered, demonstrating data remanence in versioned object storage.

Evidence



Task 8 – Lifecycle, Retention & Cryptographic Erasure

In this task, I created lifecycle rules for confidential records and incomplete uploads. I then used KMS key disablement and scheduled deletion to demonstrate cryptographic erasure.

Commands

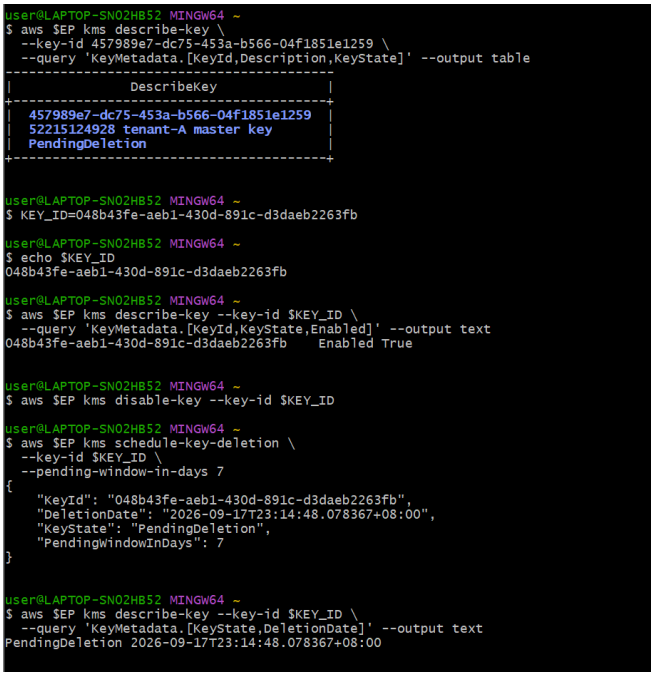
```
cat > lifecycle.json <<JSON
{
  "Rules": [
    {
      "ID": "RetireConfidentialRecords",
      "Status": "Enabled",
      "Filter": {"Prefix": "confidential/"},
      "Expiration": {"Days": 365},
      "NoncurrentVersionExpiration": {"NoncurrentDays": 30}
    },
    {
      "ID": "AbortIncompleteUploads",
      "Status": "Enabled",
      "Filter": {"Prefix": ""},
      "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7}
    }
  ]
}
JSON

aws $EP s3api put-bucket-lifecycle-configuration --bucket $BUCKET \
  --lifecycle-configuration file://lifecycle.json

aws $EP s3api get-bucket-lifecycle-configuration --bucket $BUCKET \
  --query 'Rules[].[ID,Status]' --output table

KEY_ID=048b43fe-aeb1-430d-891c-d3daeb2263fb

aws $EP kms describe-key --key-id $KEY_ID \
```

```
  --query 'KeyMetadata.[KeyId,KeyState,Enabled]' --output text

aws $EP kms disable-key --key-id $KEY_ID

aws $EP kms schedule-key-deletion --key-id $KEY_ID \
  --pending-window-in-days 7

aws $EP kms describe-key --key-id $KEY_ID \
  --query 'KeyMetadata.[KeyState,DeletionDate]' --output text
```

Result

The lifecycle configuration showed RetireConfidentialRecords and AbortIncompleteUploads as Enabled. The KMS key was initially Enabled, then disabled and scheduled for deletion. The final key state showed PendingDeletion. A later attempt to access confidential/record-v2.txt returned NoSuchKey because that object was no longer present, so that result was recorded as the actual behavior and was not treated as proof of a KMS denial.

Evidence



Data Classification Table

Classification	Who May Read It	Impact if Leaked	Control Applied
Public	Anyone	Low	Classification tag and public-access testing
Internal	Authorised staff	Medium	Least-privilege access scoped to internal/ prefix
Confidential	Authorized users only	High	Restricted bucket policy, Block Public Access and SSE-KMS

Short-Answer Questions

Q1. Which single element of the Task 2 policy caused the exposure, and why is Principal: "*" more dangerous on a bucket policy than an over-broad IAM policy attached to one user?

The element that caused the exposure was Principal: "*". It allowed anyone, including anonymous users, to access the objects. A bucket policy is more dangerous because it applies directly to the resource and can expose the bucket to everyone. In contrast, an IAM policy normally gives permissions only to the identity to which it is attached.

Q2. Explain the difference between an identity-based policy and a resource-based policy. In Task 4, which one decided each of the analyst's two requests?

An identity-based policy is attached to a user, group, or role and defines what that identity can do. A resource-based policy is attached directly to a resource such as an S3 bucket. In Task 4, the IAM policy allowed the analyst to read S3 objects, while the bucket policy allowed the internal object and explicitly denied the confidential object. An explicit Deny should take priority over an Allow.

Q3. Block Public Access is described as a guardrail rather than a control. What is the difference, and why does the distinction matter for an organization with many engineers?

A security control protects or manages access to a resource. A guardrail prevents unsafe configurations from being introduced. Block Public Access is a guardrail because it can stop engineers from accidentally making S3 data public. This matters in an organization with many engineers because many people may create or modify cloud resources.

Q4. Your bucket has default SSE-KMS encryption. Does that protect the confidential record from the analyst in Task 4? Explain precisely what server-side encryption does and does not defend against.

No. SSE-KMS protects the object at rest by encrypting it. It does not replace access control. If a request is authorized to read the object, the service can decrypt it and return the plaintext to the authorized caller. IAM and bucket policies are therefore still required to control who can access the data.

Q5. A patient invokes their right to erasure. Using your Task 7 evidence, explain why delete-object alone is not compliant, and describe two mechanisms that would make the deletion provable.

With versioning enabled, delete-object can create a delete marker while previous versions remain stored and recoverable. Therefore, delete-object alone does not prove that the data is gone. One mechanism is to delete every stored object version permanently. Another is cryptographic erasure, where the encryption key is destroyed or made unavailable so that encrypted copies can no longer be decrypted.

Q6. You are the auditor in Week 11. Name three commands from this lab whose output you would collect as compliance evidence, and state which control each one evidences.

First, get-public-access-block provides evidence that the public-access guardrail is configured. Second, head-object provides evidence that SSE-KMS encryption is applied to an object. Third, get-bucket-versioning provides evidence that S3 versioning is enabled.

Verification

The following commands verified the bucket's final security posture, including Block Public Access, versioning, default encryption, lifecycle rules, and the KMS key state.

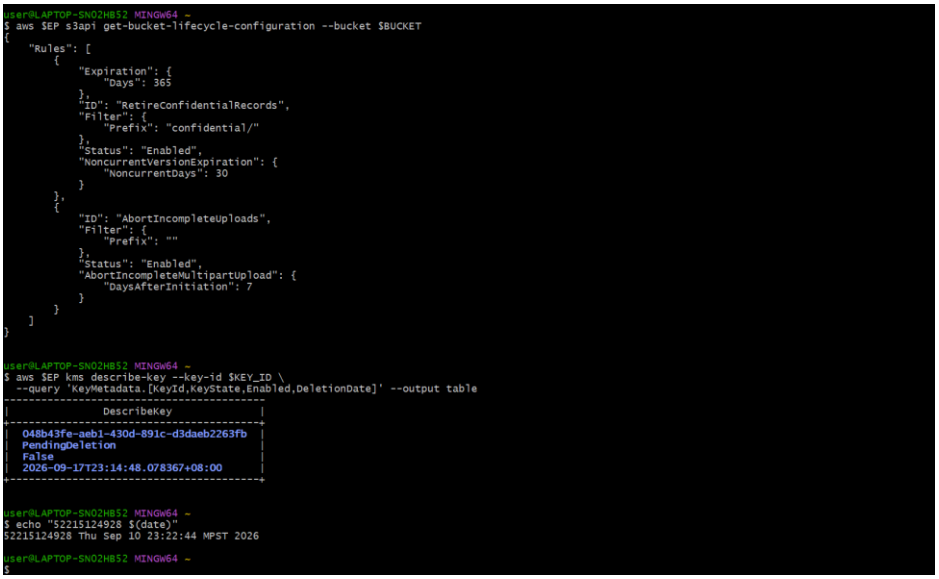
Commands

```
echo "=== IKB42603 Lab 6 verification: $BUCKET ==="

aws $EP s3api get-public-access-block --bucket $BUCKET \
  --query 'PublicAccessBlockConfiguration' --output text

aws $EP s3api get-bucket-versioning --bucket $BUCKET --output text

aws $EP s3api get-bucket-encryption --bucket $BUCKET \
  --query
'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.[SSEAlgorithm,KMSMasterK
eyID]' \
  --output text

aws $EP s3api get-bucket-lifecycle-configuration --bucket $BUCKET \
  --query 'Rules[].[ID,Status]' --output text

aws $EP kms describe-key --key-id $KEY_ID \
  --query 'KeyMetadata.KeyState' --output text
```

Result

The verification commands checked the final security settings configured during the lab. The evidence should show the Block Public Access configuration, versioning status, SSE-KMS configuration, enabled lifecycle rules, and the final KMS key state.

Evidence



Cleanup & Teardown

After completing the verification, I removed the resources created during the lab. Because the bucket used versioning, I had to remove object versions and delete markers explicitly before deleting the bucket.

Commands

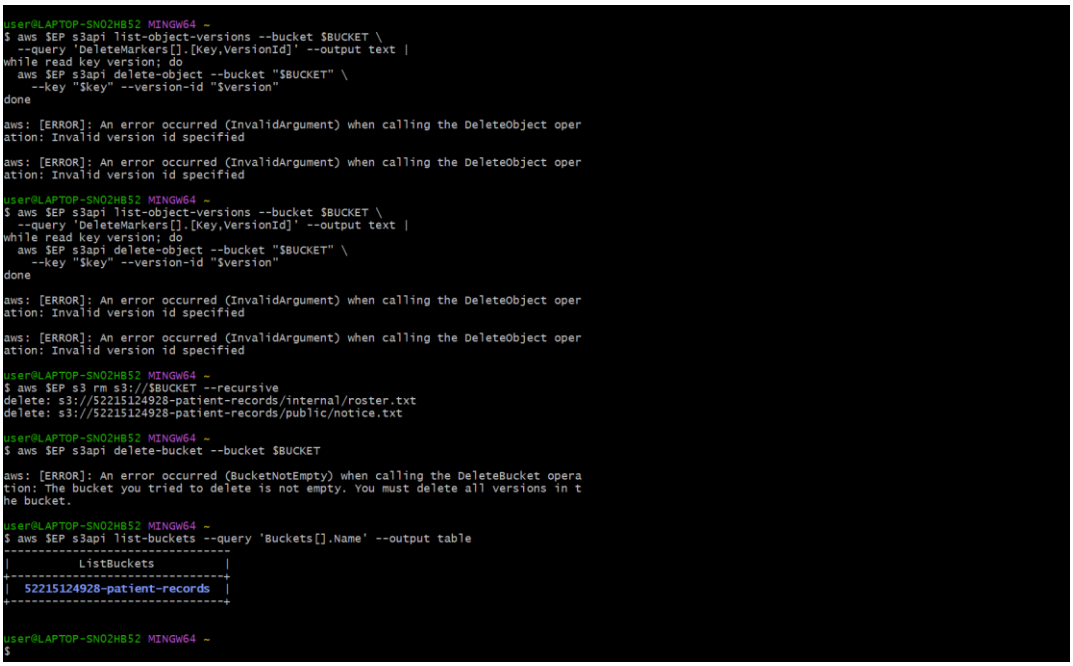
```
aws $EP s3api delete-bucket-policy --bucket $BUCKET
```

```
aws $EP s3api delete-objects --bucket $BUCKET --delete "$(aws $EP s3api \
  list-object-versions --bucket $BUCKET --output json \
  --query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}')"

aws $EP s3api delete-objects --bucket $BUCKET --delete "$(aws $EP s3api \
  list-object-versions --bucket $BUCKET --output json \
  --query '{Objects: DeleteMarkers[].{Key:Key,VersionId:VersionId}}')"

aws $EP s3api list-object-versions --bucket $BUCKET --output text

aws $EP s3api delete-bucket --bucket $BUCKET

aws $EP iam delete-user-policy --user-name DataAnalyst --policy-name S3ReadAll

aws $EP iam delete-user --user-name DataAnalyst

docker rm -f localstack

rm -f *.json *.txt
```

Result

The cleanup procedure removes all remaining object versions and delete markers, deletes the bucket and DataAnalyst IAM resources, stops and removes the LocalStack container, and removes the temporary files created during the lab.

Evidence



Security Best-Practices Checklist

☑ Data objects were classified before access decisions were applied.

☑ Public bucket exposure was demonstrated and remediated.

☑ All four Block Public Access settings were configured.

☑ Identity-based and resource-based access policies were tested.

☑ Default encryption at rest was configured using SSE-KMS.

☑ Tested a time-bounded preassigned URL.

☑ S3 versioning and delete markers were demonstrated.

☑ Lifecycle rules were configured.

☑ Cryptographic erasure was demonstrated by disabling and scheduling deletion of the KMS key.